

Particle In Cell method for collective dynamics

Simulating Plasma Instabilities and Self Gravitating Systems

M.Bourvic

Abstract

The Particle In Cell (PIC) method is a widely used class of codes for studying kinetic systems as it efficiently links the dynamics of individual particles with self-consistent field evolution, allowing for an efficient computation of many particles motion. Here we develop a PIC simulation framework in order to study collective dynamics in collisionless systems. The code is applied for the investigation of plasma instabilities and self-gravitating ensembles. In the plasma case, the system exhibits characteristic phase-space mixing, including spiral structures deriving from a two stream instability with initially perturbed velocity distributions. In the gravitational case, the simulation reproduces key features of gravitational instability including density filamentation, clumping, and eventual collapse into a dominant high-density blob. Finally, in section 4. this paper guides users on how to execute and use the code for different applications.

1. Introduction

Many-body systems exhibit a wide range of collective behaviors, that often cannot be described using local interactions only. Examples include plasmas, where electromagnetic forces couple particles over large distances and self-gravitating systems, where mass distribution evolves through a self-consistent gravitational potential. Despite their different physical origins, both systems share a common kinetic description and display similar phenomena such as instability growth, phase mixing, and spontaneous structure formation. A particularly important feature of such systems is the emergence of important

instabilities from initially small perturbations. In plasmas, these instabilities can lead to phase-space vortices, while in gravitational systems they are associated with the formation of collapsed structures.

PIC methods are regularly used to simulate such collisionless systems. They combine a Lagrangian description of particles with a Eulerian field solvers, parsing the space into discrete cells and allowing for a treatment of long-range interactions while remaining computationally efficient for a large particle number.

In this work, we develop an up to 2D PIC simulation framework to investigate collective dynamics in both plasmas and

self-gravitating systems within a unified computational approach. The goal is to reproduce and analyze key instability-driven behaviors, including phase-space mixing, filamentation, clumping, and collapse from initially near-homogeneous states. We also compare different implementations of the method using various field solvers and particle pushers, aiming for both efficiency and physical accuracy.

The focus remains qualitative and as such, all physical values used are normalized.

2. Physical Model

2.1 Generalities

We consider a system composed of a large number of particles. Each particle is defined by a charge q and mass m at a position x_p with velocity v_p and obeys Newton's law of motion.

Therefore we have along a given axis :

$$v_p = \frac{dx_p}{dt} \quad (1)$$

$$F_p = m \frac{dv_p}{dt} \quad (2)$$

Assuming these particles are collisionless, the force is derived from a self-consistent field. Sampling the space with discrete square cells, we can define a local density ρ in the cell i . The resulting potential is obtained through the Poisson equation:

$$\nabla^2 V_i = f(\rho_i) \quad (3)$$

And the field :

$$E_i = -\nabla V_i \quad (4)$$

The force is computed on the spatial grid and then interpolated back to the exact particle positions.

Finally, we assume periodic boundary conditions.

2.2 Electromagnetic case

for plasmas, we consider the charge density and follow a total of N_{part} over N_{cells} . to ensure electrical neutrality, we assume the existence of a static homogeneous background of opposite charged particles. Considering our normalized values, the local density in the cell i containing negatively charged particles $q=-1$ is given by :

$$\rho_i = \left(\frac{N_{part}}{N_{cells}} - n_i \right) / dx^2 \quad (5)$$

where n_i is the number of particles in the cell
 dx is the size of a square cell

Then, as the source term in the Poisson equation is governed by Gauss' law for electrostatics, equation (3) writes :

$$\nabla^2 V_i = -\frac{\rho_i}{\epsilon_0} \quad (6)$$

where ϵ_0 is the vacuum permittivity

Finally, the force exerted on each particles becomes :

$$F_p = qE_p \quad (7)$$

2.3 Gravitational case

for self-gravitating systems, and particle masses $m=1$, we consider the mass density. Given by :

$$\rho_i = \left(n_i - \frac{N_{part}}{N_{cells}} \right) / dx^2 \quad (8)$$

The average mass density must be subtracted to avoid singularity caused by the periodic boundary conditions.

The source term is now governed by Newton's law for gravitation, hence the following Poisson equation :

$$\nabla^2 V_i = 4\pi G \rho_i \quad (9)$$

where G is the gravitational constant

We obtain the force :

$$F_p = mE_p \quad (10)$$

3. Numerical Methods

3.1 PIC Cycle

As described in section 2.1, we use a Lagrangian description of particles with a Eulerian representation of the field in a discretized space.

At each time step, particles quantities are deposited onto the grid, field equations are solved, and the resulting field is interpolated back to particles positions, following the algorithm described in *Fig. 1*

This hybrid formulation enables the efficient simulation of collisionless systems with long-range interactions by avoiding pair interactions that would otherwise be computationally too expensive.

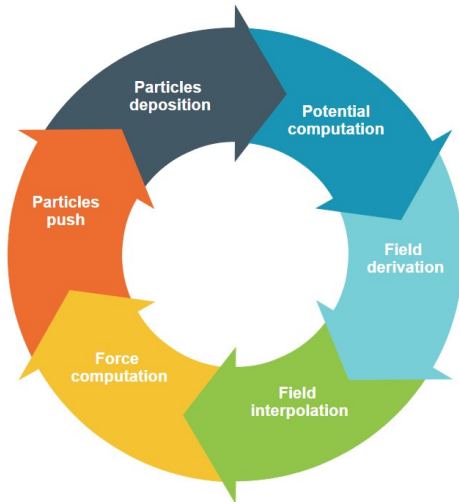


Figure 1: Particle In Cell cycle

3.2 Deposition scheme

Two different deposition techniques were investigated :

- Nearest Grid Point (NGP)
- Cloud In Cell (CIC) [1]

NGP distributes all of the particle quantities to the nearest cell while CIC takes into account particles contribution to neighbouring cells using linear weighting functions (see Appendix A)

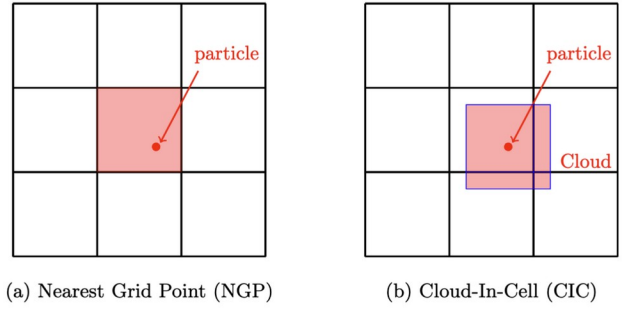


Figure 2: schematic representation of the difference between the deposition methods.
src : [alvinng4.github.io](https://github.com/alvinng4)

We show (*Fig. 3*) that CIC deposition conserves energy 3 times better than NGP by allowing for a more continuous potential and therefore less noisy field. As a consequence, CIC has been chosen as the main deposition technique in our implementation in order to mitigate numerical heating.

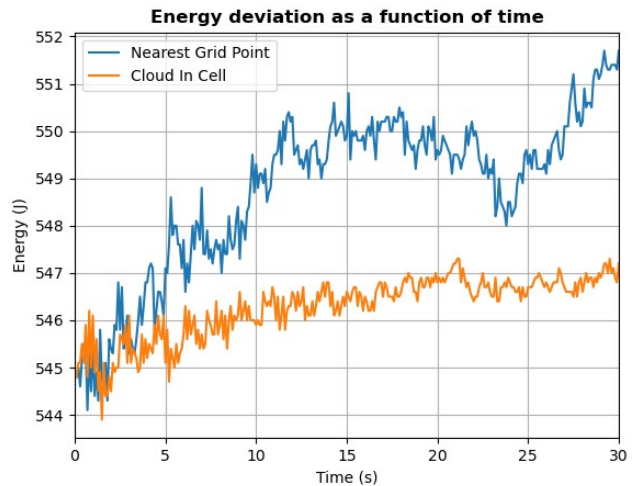


Figure 3: Energy deviation comparison between the different deposition methods using the Leapfrog scheme and Fourier solver for $1e3$ particles and a fixed time step $dt = 1e-3s$

3.3 Field solvers

The field can be computed from two alternative solvers using relaxation techniques or Fourier techniques.

In the first case, the potential obtained from equation (6) is iteratively relaxed until convergence or for a given maximum number of steps if this cannot be achieved.

Alternatively the Poisson equation can be solved in a single step by expressing it in Fourier space as:

$$-k V_k = f(\rho_k) \quad (11)$$

While both methods produce similar results, the relaxation method is prone to arbitrary energy injection when convergence is not achieved, eventually leading to non neglectable deviation over long time scale.

Bellow we plot the execution time of both solvers for a varying grid resolutions.

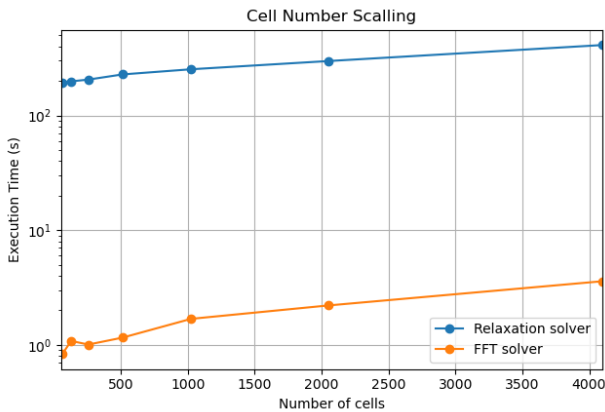


Figure 4: Execution time for a single step as a function of the number of spatial cells for both field solvers. The Benchmark was performed using the Leapfrog integration scheme discussed in section 3.4. Time step was set to $1e-3s$ and particles number to $1e3$. The maximum number of steps for the relaxation solver was limited to 500.

We demonstrate a scaling $\propto N_{cells} \log(N_{cells})$ and show that the Fourier technique executes 100 times faster than the relaxation one.

Considering its greater accuracy and rapidity, the Fourier solver should be preferred when running a simulation.

3.4 Particles integration schemes

Particles trajectories are integrated using four different schemes :

- Explicit Euler
- Implicit Euler
- Leapfrog
- Range-Kutta 4

Specific update equations are explicated in Appendix B

3.5 Time integration

In addition to fixed time-stepping, an adaptive time-stepping option has been implemented following the Courant-Friedrichs-Levy (CFL) condition :

$$\Delta t = \frac{dx}{\max(v_p)} \quad (12)$$

with

dx the size of a cell

$\max(v_p)$ the maximum velocity among particles

this ensure particles won't cross more than a cell during a given time step, ensuring numerical stability.

3.6 Validation

The accuracy of the integration schemes is assessed from energy conservation

Explicit Euler method displays (Fig. 5A) a significant energy gain converging towards a limit more than 300 times the initial energy. This behavior is expected as this scheme is non symplectic and computes position from a non-updated velocity. Other symplectic schemes conserve energy to within 3% over the course of a 10s evolution. The numerical heating, while limited with a fixed time step, scale proportionally to the evolution time, eventually leading to significant inaccuracies. This can be solved by introdu-

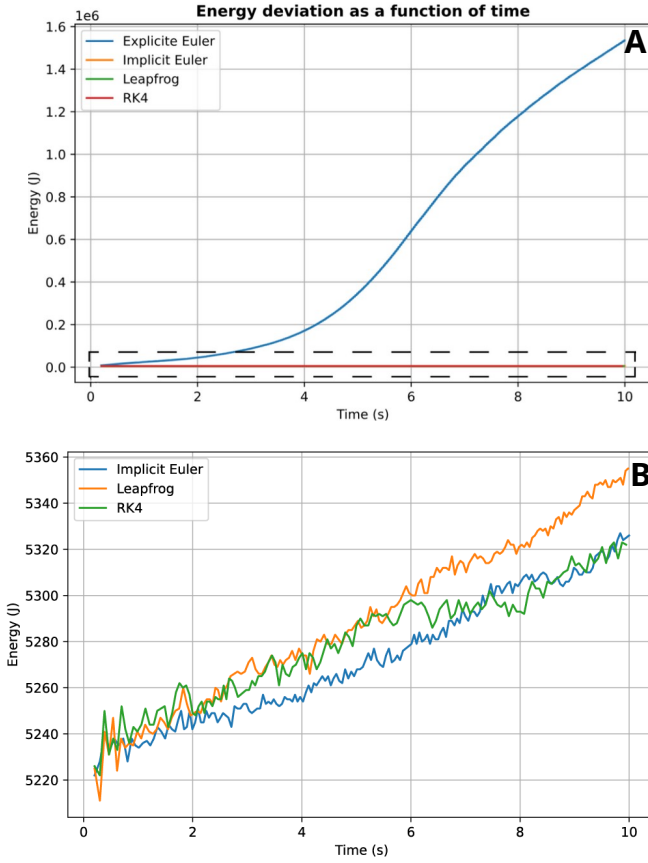


Figure 5: **A.** Energy deviation as a function of time for the different integration schemes. Particles number is set to $1e4$ and fixed time-stepping is used with $dt = 1e-3$ s. **B.** Zoom over the dotted area in panel A.

-cing the variable time step criterion from 3.5. Comparison of energy deviation with and without variable time-stepping is plotted below.

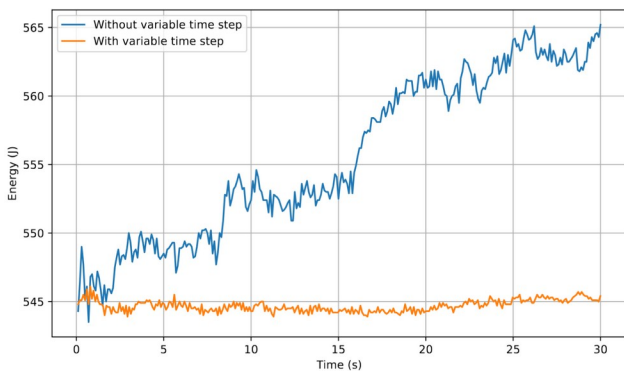


Figure 6: Energy conservation with and without variable time-stepping for the Leapfrog algorithm. Particles number is set to $1e3$ and fixed time step to $1e-3$ s.

Variable time-stepping ensures energy conservation to within 0.2%, allowing for long time evolution with accurate results.

3.7 Performance analysis

Performance is evaluated in terms of runtime and scaling with respect to the number of particles N_{part}

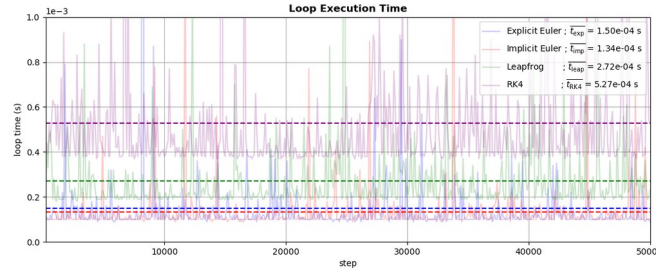


Figure 7: Single time step execution time for the different integration schemes. Fourier solver is used for $1e3$ particles and a fixed time step $dt = 1e-3$ s

We show (Fig. 7) single step execution time increasing up to 3.5 times between methods without significant gain in accuracy (Fig. 5). The different integration techniques scale as $\propto N_{part}$ (Fig. 8) causing the number of particles to be the main limitation when one wants to preserve reasonable computation time.

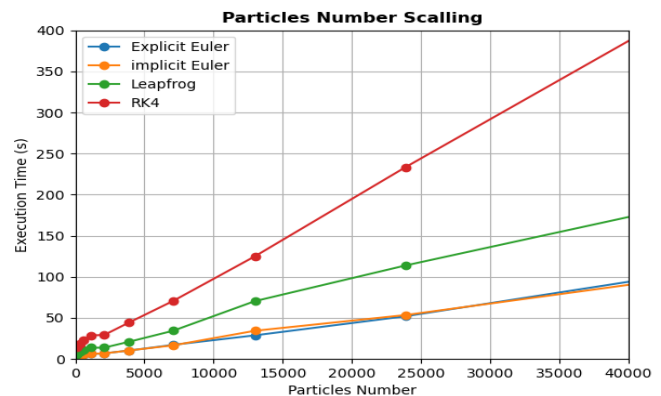


Figure 8: Total execution time as a function of the number of particles. Fourier solver is used with a fixed time step $dt = 1e-3$ s

4. Code structure and execution

The project directory contains two main folders corresponding to different versions of the code :

- 1D Electromagnetic
- 2D Electromagnetic & gravitational

Structure within these folders is not modified.

Disclaimer: The 2D version being the ongoing one for development, some features may be lacking or deprecated in the 1D Electromagnetic directory. 2D is configured by default for gravitational systems but can be modified to electromagnetism by removing `_grav` in the module imports

`requirements.txt` lists the required libraries. All should be all available from Spyder with the exception of `tqdm`. Installation can be done in the Spyder console by running

> `conda install tqdm`

The program can then be executed by running `main.py`

Configuration is either done manually from `modules/config.py` and `modules/config_grav.py` or by starting from a previously saved file. Included parameters, their type and a brief description are listed in table 1.

More refined settings of the initial conditions (spatial and velocity distributions, ...) can be found in `modules/init.py` and `modules/init_grav.py`

Variable Name	Type	Default Value	Description
Part_num	Integer	1000	Total number of particles
q	Float or array	-1.0	Particles electric charge
m	Float or array	1.0	Particles mass
Pos_disp	Float	0.1	Initial spatial dispersion
Vel_disp	Float	0.1	Initial velocity dispersion
L	Float	1.0	Total physical length of the simulation domain
Cell_num	Integer	300	Number of grid cells (field resolution)
eps0	Float	1.0	Normalized vacuum permittivity constant
Tmax	Float	30.0	Maximum physical simulation runtime
dt0	Float	1e-3	Fixed time step
Relax_step_max	Integer	50	Max iteration for the relaxation solver.
epsilon	Float	1e-4	Convergence tolerance for the relaxation solver.
save_step	Integer	100	Frame interval at which simulation data is saved
random_seed	Integer	0	Random seed allowing reproductibility

Table 1: configuration parameters

When running the program, the user will be prompted to select if the simulation has to be resumed from a previous save, which solver and scheme to use, if variable time step is required and if the result is to be saved.

Other documents in `modules/` contain the functions used by the program, sorted by category. Names of the files should be self explanatory.

Saves are located in `/data`, to a folder named by the user during the prompt. Included files are :

File	Description
name_main.dat	An ASCII file containing all the variable used for the simulation and the energy at each saved time step.
name_position.dat	A binary file containing the position of each particle at each saved time step
name_velocity.dat	A binary file containing the velocity of each particle at each saved time step
name_potential.dat	A binary file containing the potential in each cell at each saved time step
name_resume.npz	A Numpy binary archive containing the last position, velocity and potential for each particle and cell.

Table 2: Saved files description

***Disclaimer:** Resuming from a previously computed file may lead to unexpected results as the feature is not fully debugged*

5. Physical results

5.1 Plasma Dynamics

Here we investigate the canonical case of two streams instabilities in 1D plasmas.

5.1.1 Initialization

The system is initialized with two streams (later denoted as Populations A and B) of electrons moving opposite to one another and against a uniform neutralizing ions background. At the beginning of the simulation, the streams are centered around respective initial velocities $+v_0$ and $-v_0$. Individual particles velocities are modulated around these values by a thermal velocity dispersion as defined by v_{disp} . Positions are seeded in the domain from a uniform random distribution. The small scale asymmetry of the system is required to trigger instabilities.

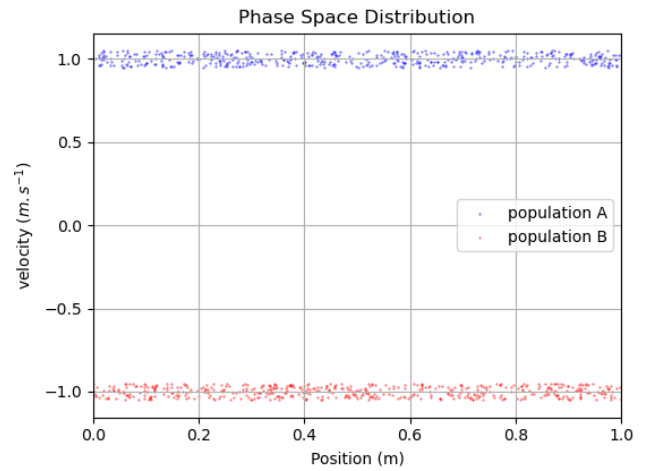


Figure 9: Initial phase space distribution. Populations A and B are well defined.

5.1.2 Phase space evolution

The macroscopic evolution of the system is best understood by looking at the position-velocity phase space distribution.

Initially, the two streams pass through

one another however the non-uniform spatial distribution breaks charge neutrality and gives rise to an electrostatic potential. The subsequent field generates a feedback loop where particles are accelerated in some regions and decelerated in others. The initially small perturbation grows exponentially.

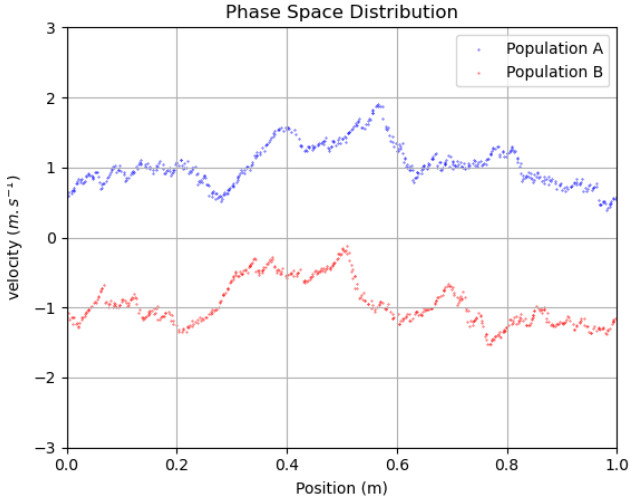


Figure 10: Growth of the instability

As the potential wells grow sufficiently deep, they begin to trap particles into spiral structures, effectively mixing the two streams.

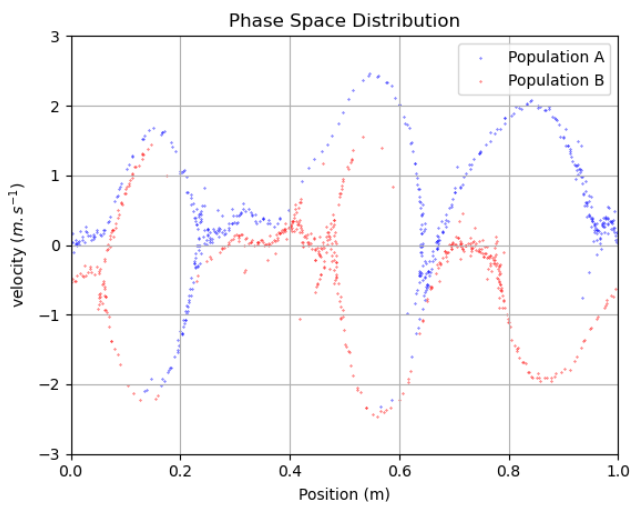


Figure 11: Spiral instabilities in the phase space

At late simulation times, the vortices begin to shear and break apart due to filamentation and mixing. Because the system is collisionless, the entropy increase is driven entirely by this self-consistent phase mixing. The initially distinct streams become impossible to discriminate and only remains a broad and stable velocity distribution.

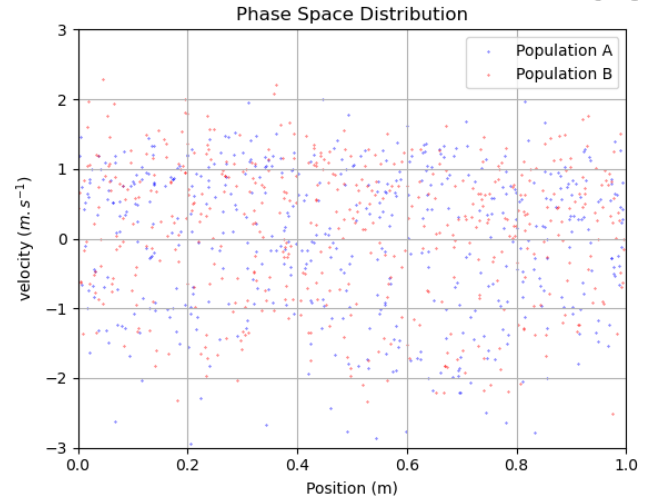


Figure 12: Heated plasma from the initial two streams. Populations cannot be spatially identified anymore

5.2 Gravitational dynamics

Here we get interested into dynamics of collisionless gravity-dominated systems in 2D. Evolution is dictated by the Jeans criterion. Gravitational attraction and velocity dispersion compete and govern the system's dynamics.

5.2.1 Initialization

Particles are randomly sampled over the whole domain (*Fig. 13*) and are given a rotational velocity around the initial barycenter of the system. Individual velocities are modulated by a thermal velocity dispersion similar to section 5.1.1.

5.2.2 Filamentation and clumping

Very soon after the simulation begins, the random spatial distribution collapses and forms a network of high density string-like structures (*Fig. 14*) known as filaments.

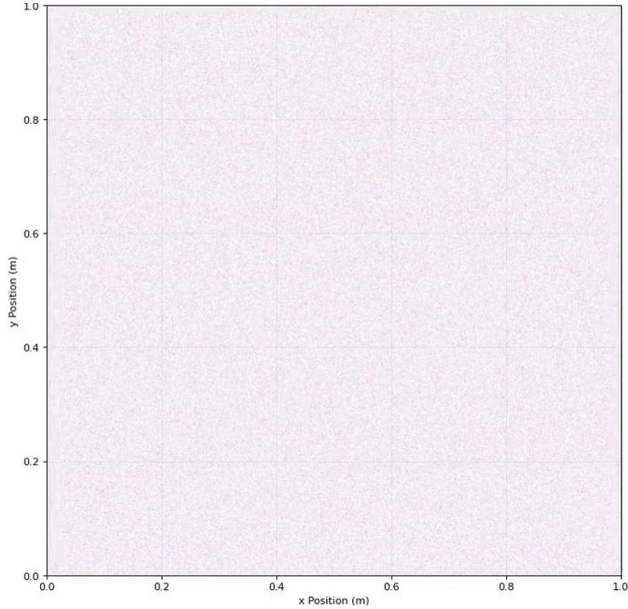


Figure 13: Initial distribution of particles. The space is filled homogeneously

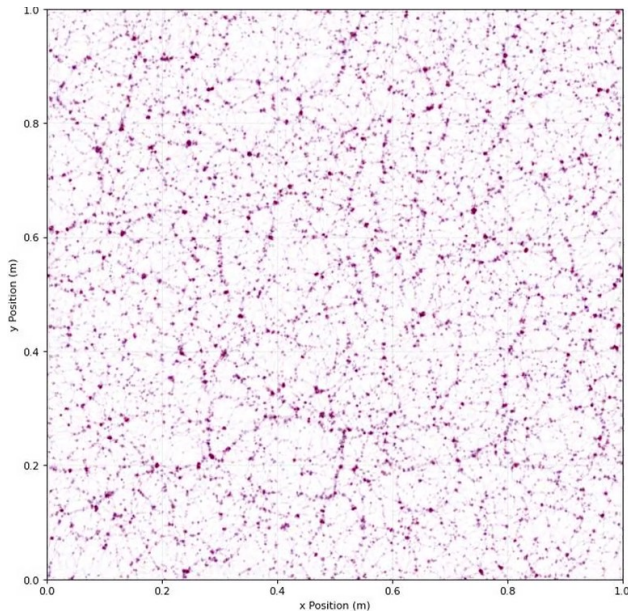


Figure 14: Filaments after collapse has started. Some clumps can already be observed, interconnected by the filaments.

Once the filaments form, matter flows longitudinally along these structures

toward local intersections. This fragmentation process breaks the filaments apart into distinct isotropic clumps. Over long timescales, these clumps gravitationally attract one another and progressively merge.

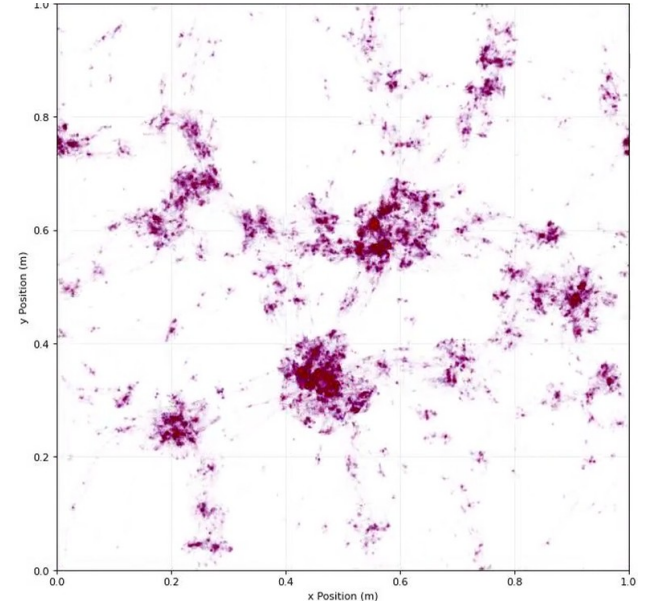


Figure 15: Large clumps resulting from filaments accretion

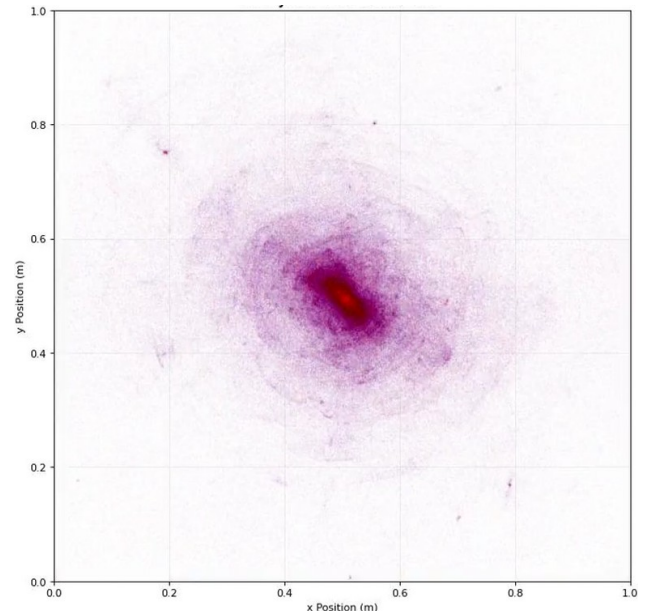


Figure 16: Dense central structure with a peripheral halo of lesser density. Satellite clumps remain orbiting and will eventually be torn apart when approaching the massive structure.

5.2.3 Virialization

The final structure does not collapse down to an infinitely small point. Instead, as particles fall into the gravitational potential well, they gain an incredible amount of kinetic energy. The system eventually achieves a state of dynamic equilibrium known as Virialization where the thermal pressure keeps particle from falling to a singularity.

In this state we verify :

$$2K + U = 0 \quad (13)$$

with K the kinetic energy

U the gravitational potential energy

6. Discussion

The simulations highlight the strong similarities that can be observed between plasmas and gravitational ensembles despite the fundamentally different nature of the underlying forces. In both cases, the dynamics are governed by collective interactions rather than pair interactions, leading to the spontaneous amplification of initially small perturbations.

One of the most striking aspects of the plasma simulations is the formation of spiral structures in the phase space during the evolution of the two-stream instability. These structures result from phase mixing progressively distorting the distribution function.

The gravitational simulation exhibit qualitatively similar behavior. Small density perturbations evolve into elongated filamentary structures before collapsing into localized high-density regions. The appearance of hierarchical clumping shows that even simplified collisionless models are capable of reproducing essential mechanisms

associated with gravitational structures formation. Although the simulation remains highly idealized, it demonstrates how self-gravity alone can drive organization of large-scale structures from nearly homogeneous initial conditions.

The simulations additionally emphasize several intrinsic limitations of PIC methods. Because of the limited number of particles, statistical noise is unavoidable and becomes increasingly significant at low particle counts. Spatial discretization also introduces supplementary noise through the derivation of a field from a discontinuous potential. Increasing particles number and grid resolution improves accuracy but rapidly increases computational cost.

Finally, while the present work focuses on collisionless dynamics in simplified geometries, the framework naturally opens the possibility of more advanced extensions. In particular, higher-dimensional simulations and improved physical models could allow for the study of more realistic systems and more complex collective behaviors. The current implementation therefore serves both as a validation of the PIC approach for instability-driven dynamics and as a foundation for future developments in kinetic simulations of long-range interacting systems.

7. Conclusion

In this work, we developed a Particle In Cell simulation framework dedicated to the study of collective dynamics in collisionless systems governed by long-range interactions. The project focused on two distinct but

mathematically related regimes: plasma instabilities and self-gravitating systems. By relying on a unified kinematic description, the code was able to reproduce characteristic behaviors emerging from initially perturbed or near-homogeneous particle distributions

For plasmas, the simulation successfully reproduced the evolution of the two-stream instability, including the formation of spiral structures in the phase space and the progressive mixing of the particle distribution.

In the gravitational case, the simulation reproduced the development of Jeans instabilities, leading to density filamentation, clumping, and eventual collapse into high-density structures.

Beyond the physical results, particular attention was given to the numerical aspects of the implementation. Different field solvers and integration schemes were compared in terms of accuracy, numerical stability, and computational cost. The implementation of adaptive time-stepping using a CFL condition allowed improved stability. Performance analyses highlighted the trade-offs between numerical precision and computational expense for the different algorithmic choices.

Although the present work remains limited to collisionless dynamics and simplified planar problems, it provides a flexible computational framework for studying collective phenomena in systems with long-range interactions. Possible future extensions include 3-dimensional simulations and the implementation of effective transport or

viscosity models inspired by astrophysical disk dynamics.

Overall, this project demonstrates how PIC methods provide a powerful and versatile approach for investigating instability growth, phase-space evolution and spontaneous structure formation in both plasmas and gravitational systems.

Acknowledgment

I thank Pr. Dominique Aubert for tutoring this work and his helpful advice

References

[1] C. K. Birdsall and D. Fuss, J. Comput. Phys. 3, 494 (1969). DOI: 10.1016/0021-9991(69)90058-8.

Appendix

A. Cloud-In-Cell weighting functions

The Cloud-In-Cell (CIC) deposition scheme distributes the quantities carried by a particle (charge and/or mass) over the four closest nodes of the 2D spatial grid, ensuring a more continuous density field and mitigating numerical grid heating.

For a particle p located at continuous spatial coordinates (x_p, y_p) within a uniform grid of square cells of size dx , we first define the indices (i_p, j_p) of the bottom-left node of the cell bounding the particle from the floor function:

$$i_p = \lfloor \frac{x_p}{dx} \rfloor \quad j_p = \lfloor \frac{y_p}{dx} \rfloor$$

We then define the normalized local coordinates of the particle within this specific cell, such that :

$$pos_x = \frac{x_p}{dx} - i_p \quad pos_y = \frac{y_p}{dx} - j_p$$

Finally, the total quantities of the particle is distributed to the four surrounding cells using the linear weighting functions summarized in the following Table :

Grid Node	Index	Weighting Function
Bottom-Left	(i_p, j_p)	$W_{bl} = (1 - pos_x)(1 - pos_y)$
Bottom-Right	(i_{p+1}, j_p)	$W_{br} = pos_x(1 - pos_y)$
Top-Left	(i_p, j_{p+1})	$W_{tl} = (1 - pos_x)pos_y$
Top-Right	(i_{p+1}, j_{p+1})	$W_{tr} = pos_x pos_y$

Table 1: CIC node assignment and corresponding weighting functions

These weighting functions are then reused for the interpolation of the field back to particles.

B. Integration schemes

To evaluate the numerical stability and performance of the PIC framework (Sections 3.6 and 3.7), four distinct integration schemes were implemented. Their core computational properties are compared in Table A2.

Scheme	Order	Symplectic	Field eval/step
Explicit Euler	1	No	1
Semi-Implicit Euler	1	Yes	1
Leapfrog	2	Yes	1
Runge-Kutta 4	4	No	4

Table 2: Numerical properties of the implemented particle pushers

Let x_n and y_n denote the position and velocity of a particle at time step n . Time steps are separated by an increment Δt .

Let $a_n(x_n) = \frac{F(x_n)}{m}$ represent the field-derived acceleration at a specific coordinate.

The discrete update rules for each scheme are defined below.

1. Explicit Euler

$$v_{n+1} = v_n + a(x_n) \Delta t$$

$$x_{n+1} = x_n + v_n \Delta t$$

2. Implicit Euler

$$v_{n+1} = v_n + a(x_n) \Delta t$$

$$x_{n+1} = x_n + v_{n+1} \Delta t$$

3. Leapfrog

$$x_{n+1} = x_n + v_n \Delta t + \frac{1}{2} a(x_n) \Delta t^2$$

$$a_{n+1} = a(x_{n+1})$$

$$v_{n+1} = v_n + \frac{1}{2} (a(x_n) + a_{n+1}) \Delta t$$

4. Range-Kutta 4

$$\begin{aligned}
k_{1,x} &= v_n & k_{1,v} &= a(x_n) \\
k_{2,x} &= v_n + \Delta \frac{t}{2} k_{1,v} & k_{2,v} &= a\left(x_n + \Delta \frac{t}{2} k_{1,x}\right) \\
k_{3,x} &= v_n + \Delta \frac{t}{2} k_{2,v} & k_{3,v} &= a\left(x_n + \Delta \frac{t}{2} k_{2,x}\right) \\
k_{4,x} &= v_n + \Delta t k_{3,v} & k_{4,v} &= a\left(x_n + \Delta t k_{3,x}\right) \\
x_{n+1} &= x_n + \Delta \frac{t}{6} (k_{1,x} + 2k_{2,x} + 2k_{3,x} + k_{4,x}) \\
v_{n+1} &= v_n + \Delta \frac{t}{6} (k_{1,v} + 2k_{2,v} + 2k_{3,v} + k_{4,v})
\end{aligned}$$